QUIZ PERTEMUAN 7

PEMROGRAMAN BERORIENTASI OBJEK

Oleh:

Zeyan Fahluzi Alhariz

2506602201033

SOAL

1. Kosep Procedural Programming

Jelaskan pengertian procedural programming dan bagaimana data dikelola pada pendekatan tersebut.

Jawaban:

- Pengertian: *Procedural programming* adalah paradigma pemrograman yang berfokus pada penulisan serangkaian instruksi langkah demi langkah (berupa prosedur atau fungsi) yang dieksekusi dari atas ke bawah untuk menyelesaikan suatu tugas.
- Pengelolaan Data: Pada pendekatan ini, data dikelola secara terpisah dari fungsi yang mengoperasikannya. Data umumnya disimpan menggunakan variabel atau struktur data dasar (seperti *list* atau *dictionary*) dan dilempar (*passed*) ke dalam fungsi-fungsi tersebut sebagai argumen.

2. Perbedaan Pendekatan

Jelaskan perbedaan utama antara procedural programming dan OOP dalam hal penyimpanan data dan pengorganisasian kode.

Jawaban:

- Procedural Programming: Memisahkan antara data dan perilaku (fungsi). Pengorganisasian kode berpusat pada "tindakan" atau fungsi operasional (misal: `tambah_buku()`, `hapus_buku()`) yang memanipulasi sekumpulan data yang bersifat pasif.
- Object-Oriented Programming (OOP): Menggabungkan data (atribut) dan perilaku (*method*) ke dalam satu kesatuan entitas yang disebut "objek" (*encapsulation*). Pengorganisasian kode berpusat pada entitas dunia nyata (seperti *class* `Buku` atau `Anggota`) yang dapat berinteraksi satu sama lain.

3. Analisis Kode

```python
buku = {"judul": "Pemrograman Python", "tersedia": True}

def pinjam_buku(buku):
    if buku["tersedia"]:
        buku["tersedia"] = False
        return True
    return False
```

Identifikasi pendekatan yang digunakan pada kode tersebut dan jelaskan alasan teknisnya.

Jawaban:

- Identifikasi Pendekatan: Kode tersebut menggunakan pendekatan Procedural Programming.
- Alasan Teknis: Kode tersebut memisahkan data dan perilakunya. Data status buku disimpan dalam sebuah struktur *dictionary* (`buku = {...}`), sedangkan perilakunya dijalankan oleh fungsi independen (`def pinjam_buku(buku):`). Fungsi tersebut menerima *dictionary* sebagai argumen untuk dimanipulasi, alih-alih data tersebut berada di dalam satu objek mandiri.

4. Kapan Menggunakan OOP

Sistem informasi oerpustakaan akan dikembangkan dengan menambah jenis koleksi baru dan jenis pengguna baru. Jelaskan mengapa OOP lebih tepat digunakan untuk kebutuhan tersebut.

Jawaban:

OOP jauh lebih tepat digunakan ketika sistem mulai kompleks karena menyediakan konsep Inheritance (pewarisan) dan Polymorphism. Ketika Sistem Informasi Perpustakaan perlu menambah jenis koleksi (misal: Jurnal, Majalah) dan jenis pengguna (misal: Dosen, Mahasiswa, Staf), *developer* cukup membuat satu *superclass* (seperti `Koleksi` atau `Pengguna`), lalu menurunkan atribut dan *method* utamanya ke *subclass* yang lebih spesifik. Hal ini mencegah duplikasi kode, menjaga kerapian arsitektur, dan mempermudah pemeliharaan seiring bertambahnya fitur.

5. Analisis Kebutuhan

Tuliskan tiga pernyataan analisis kebutuhan yang membantu menentukan apakah sebuah program sebaiknya menggunakan OOP atau procedural programming. Jelaskan jawaban yang mengarahkan pada OOP.

Jawaban:

Tiga Pertanyaan Analisis Kebutuhan:

1. Apakah sistem ini mensimulasikan entitas dunia nyata (seperti Pengguna, Kendaraan, Produk) yang masing-masing memiliki atribut dan tindakan yang terikat?
2. Apakah entitas-entitas di dalam program memiliki hierarki hubungan (seperti hubungan *is-a* / "adalah sebuah") yang membutuhkan spesifikasi atau pewarisan?
3. Apakah program dirancang untuk proyek skala besar yang berpotensi terus dikembangkan fungsinya dalam jangka panjang dengan modifikasi data yang kompleks?

Jawaban yang Mengarah pada OOP: Jika jawaban terhadap pertanyaan di atas mayoritas adalah "Ya", maka pendekatan yang harus dipilih adalah OOP. Menjawab "Ya" berarti sistem membutuhkan abstraksi (*abstraction*), pembungkusan status data yang aman (*encapsulation*), serta penggunaan ulang kode melalui pewarisan hierarki (*inheritance*), yang mana seluruh fitur tersebut merupakan kekuatan utama dari *Object-Oriented Programming*.